LIMITBEYOND

FITNESS & GYM MANAGEMENT PLATFORM

Engineering Standards & Implementation Guide

Version 1.0 · How code is written, reviewed and enforced

Document ID	**LB-ENG-1.0**
Status	**Draft for review**
Date	**16 September 2026**
Prepared by	**Bibek Jha**
Implements	**LB-SRS-1.1 · LB-ARCH-1.0**
Applies to	**Backend, mobile app, web console, CI**
Companion file	`CLAUDE.md` **— Appendix C**

> **How to use this document.** The SRS said what to build; the architecture said how to shape it. This says how to write it, so that a module built in Phase 4 looks like one built in Phase 1 and the security properties hold in both. Rules marked AUTOMATED are enforced by tooling in CI — those are the ones that will still be true in a year. Rules that rely on discipline are kept few on purpose.

Table of Contents

1. **Principles and How Rules Are Enforced**
2. **Repository and Project Structure**
3. **Backend Standards — Java and Spring**

   3.1 Layering · 3.2 Module boundaries · 3.3 Reference implementations · 3.4 Transactions · 3.5 Errors · 3.6 Validation · 3.7 Nullability and immutability
4. **The Five Non-Negotiables**

   The rules that, if broken, silently produce a security or correctness defect
5. **API Design Conventions**
6. **Database Standards**
7. **Security Coding Standards**
8. **Mobile Standards — React Native and Expo**
9. **Testing Strategy**
10. **Automated Quality Gates**
11. **Git Workflow and Code Review**
12. **Observability Standards**
13. **The Permission Matrix**
14. **Appendix A — Code Review Checklist**
15. **Appendix B — Definition of Done**
16. **Appendix C — `CLAUDE.md` Agent Ruleset**
17. **Appendix D — SRS Amendments Required**

1. Principles and How Rules Are Enforced

1.1 Why this document is short on style and long on safety

Most of what is usually called a coding standard — brace placement, import order, line length — is settled here by running a formatter and never discussing it again. Spotless and Prettier reformat on commit; nobody reviews whitespace. What remains is the small set of rules where getting it wrong produces a defect that testing will not catch and a reviewer will not notice.

This system has an unusual concentration of those. Row-Level Security fails *open* when misconfigured, which means a broken isolation boundary looks exactly like a working one until a gym sees another gym's members. A media upload with the wrong delivery type produces a permanent public URL for a member's receipt or identity document. A `SET` where `SET LOCAL` belonged leaks tenant context between pooled connections. None of these throw an exception. All of them pass a happy-path test.

Section 4 collects those into five rules. If you read nothing else in this document, read that.

1.2 Rule categories

Marking	Meaning
AUTOMATED	Enforced by a tool in CI. Violating it fails the build. Preferred wherever a rule can be expressed mechanically — a convention nothing enforces decays within months.
REVIEW	Checked by a human at pull request. Kept deliberately few, because attention is finite and spending it on formatting means not spending it on authorisation.
CRITICAL	A rule whose violation is a security or data-integrity defect. These are both automated and reviewed, and they are the only rules in this document that may never be waived for expedience.

1.3 Principles

ID	Principle	In practice
ENG-01	Make the wrong thing hard to write	Prefer a design where the unsafe path does not compile or does not exist over one where it is merely discouraged. A `MediaService` that requires a sensitivity argument beats a comment asking people to remember.
ENG-02	Automate the rule or drop it	If a convention cannot be machine-checked and is not security-critical, it is a preference, not a standard. Write it down only if it earns the attention.
ENG-03	Explicit over clever	This codebase will be read by its author months later and by an AI agent with no memory. Obvious code with a boring name beats a concise idiom that needs explaining.
ENG-04	The test suite is the specification	Requirements traceability lives in test names. A test called `locationAdminCannotReadSiblingLocationMembers` is the executable form of `NFR-SEC-01a` and is what makes the claim defensible.
ENG-05	One way to do each thing	One error model, one pagination shape, one way to acquire tenant context, one way to enqueue a job. Variation without reason is a cost with no benefit.
ENG-06	Fail closed, loudly	Missing authorisation context yields zero rows and an error, never an unfiltered query. Ambiguity resolves toward refusal.

2. Repository and Project Structure

2.1 One repository

A single repository holds the backend, the mobile app, the web console and infrastructure. The decisive reason is the generated API client: backend and clients change together, and a monorepo lets one pull request carry an endpoint change and the regenerated client that consumes it. Split repositories would make that two coordinated merges and an interval during which they disagree.

```
limitbeyond/
├── backend/                  // Spring Boot, Maven
├── mobile/                   // React Native + Expo
├── web/                      // React + Vite admin console
├── packages/api-client/      // GENERATED from OpenAPI — never hand-edited
├── infra/                    // Dockerfiles, compose, deploy scripts
├── docs/                     // SRS, architecture, this document, ADRs
├── .github/workflows/        // CI pipelines
└── CLAUDE.md                 // agent ruleset — Appendix C
```

2.2 Backend package structure

Packages follow the ten architectural modules of LB-ARCH-1.0 §4.3, not technical layers. A module owns its controllers, services, entities and repositories together; there is no global `controllers` package. This is what lets Spring Modulith verify boundaries at build time.

```
com.limitbeyond
├── platform/      identity/      org/         people/      billing/
├── attendance/    scheduling/    training/    engagement/  insights/
└── shared/        // cross-cutting only: audit, error model, RLS context, job queue

// Inside every module, the same shape:
attendance/
├── AttendanceModule.java        // @ApplicationModule — declares allowed dependencies
├── api/                         // PUBLIC: what other modules may use
│   ├── AttendanceFacade.java    //   interface, the only entry point
│   └── CheckInRecorded.java     //   published event (record)
├── web/                         // package-private: controllers, request/response DTOs
├── domain/                      // package-private: entities, value objects, invariants
├── internal/                    // package-private: service implementations
└── persistence/                 // package-private: repositories
```

Rule ENG-2.1 — Only `api/` is public AUTOMATED

Every class outside a module's `api/` package is package-private. A module reaching into another's `domain`, `internal` or `persistence` fails the Modulith verification test and the build. **Why:** boundaries decay silently under delivery pressure, and restoring them later costs far more than maintaining them. Making the compiler the enforcer removes the judgement call entirely.

2.3 Mobile structure

```
mobile/
├── app/                    // expo-router: file-based routes only, no logic
│   ├── (auth)/  (training)/  (member)/  (trainer)/  (admin)/  (superadmin)/
│   └── checkin/
├── features/               // feature modules: hooks, components, local logic
│   ├── attendance/  workouts/  progress/  bookings/  billing/
├── lib/
│   ├── api/                // re-exports packages/api-client + interceptors
│   ├── db/                 // expo-sqlite + Drizzle: schema, outbox
│   ├── sync/               // outbox drain, retry, idempotency
│   ├── auth/               // secure token storage, refresh, grant-aware nav
│   └── media/              // Cloudinary upload + URL construction
├── components/ui/          // design-system primitives, no business logic
└── theme/
```

Rule ENG-2.2 — Route files contain no logic REVIEW

A file under `app/` renders a screen component from `features/` and nothing else — no data fetching, no business logic, no conditionals beyond a permission guard. **Why:** routes are the hardest part of the app to test; keeping them empty makes the testable part the part that matters.

3. Backend Standards — Java and Spring

3.1 Layering

Three layers, one direction of travel: **web** → **service** → **persistence**. Skipping a layer is not permitted, and neither is travelling back up.

Layer	May do	Must never do
web	Deserialise and validate input, call exactly one service method, map the result to a response DTO, set the status code.	Touch a repository. Contain business rules. Open a transaction. Decide authorisation.
service	Enforce authorisation, own the transaction boundary, apply business rules, orchestrate repositories, publish events, enqueue jobs.	Reference `HttpServletRequest`, response DTOs, or any web type. Return entities outside the module.
persistence	Query and persist. Own no logic beyond query construction.	Call a service. Perform authorisation. Be referenced from `web`.

Rule ENG-3.1 — Authorisation lives in the service layer CRITICAL

Every service method that reads or writes tenant data begins with an authorisation check naming the permission and the location it applies at. Not in the controller — controllers are added, forgotten and duplicated; a service method is the single chokepoint every caller passes through. **Why:** `FR-AUTH-28` requires authority be evaluated *per location*. One central evaluation is auditable; scattered annotations are not.

3.2 Reference implementation — controller

```
@RestController
@RequestMapping("/api/v1/locations/{locationId}/check-ins")
@RequiredArgsConstructor
class CheckInController {                       // package-private: nothing outside needs it

    private final AttendanceFacade attendance;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    CheckInResponse record(@PathVariable LocationId locationId,
                           @Valid @RequestBody CheckInRequest request) {
        // No logic. No authorisation. No repository. One call, one mapping.
        return CheckInResponse.from(
            attendance.recordCheckIn(locationId, request.toCommand()));
    }
}
```

3.3 Reference implementation — service

```java
@Service
@RequiredArgsConstructor
class CheckInService implements AttendanceFacade {

    private final AttendanceRepository attendanceRepo;
    private final MembershipFacade memberships;      // another module's api/ — allowed
    private final JobQueue jobs;
    private final AuthContext auth;

    @Override
    @Transactional                                    // the ONLY place a transaction starts
    public CheckIn recordCheckIn(LocationId locationId, RecordCheckInCommand cmd) {

        auth.require(Permission.RECORD_CHECK_IN, locationId);   // ENG-3.1, always first

        // Idempotency before side effects — a replayed offline submission
        // must return the original result, not create a second row (FR-ATT-05).
        Optional<CheckIn> existing =
            attendanceRepo.findByUserAndIdempotencyKey(cmd.userId(), cmd.idempotencyKey());
        if (existing.isPresent()) return existing.get();

        Membership membership = memberships.activeFor(cmd.userId(), locationId)
            .orElseThrow(() -> new MembershipNotActiveException(cmd.userId()));

        // Domain invariants belong to the domain object, not to this method.
        CheckIn checkIn = CheckIn.record(cmd, membership, auth.currentUser());

        attendanceRepo.save(checkIn);

        // Enqueued IN THIS TRANSACTION: a rollback removes the job too (ADR-09).
        jobs.enqueue(new RecalculateStreak(cmd.userId()));

        return checkIn;
    }
}
```

3.4 Transactions

Rule	Reason
`@Transactional` on service methods only	Controllers and repositories must not define transaction scope. One boundary per business operation keeps the atomicity story legible.
Never call an external service inside a transaction	Twilio, Cloudinary and Expo calls hold a database connection open for the duration of a network round trip. Enqueue a job instead; the job runs after commit.
Use `lockForUpdate` for contended rows	Session capacity and credit balances are contended. A read-then-write without a lock is a lost update under concurrency, which is exactly what `NFR-PRF-05` forbids.
Read-only operations use `@Transactional(readOnly = true)`	Signals intent and lets the driver optimise. It is also where the RLS context is set, so even reads must be transactional.

3.5 Error handling

One exception hierarchy, one response shape. Errors carry a stable machine-readable code; messages are for humans and may change without notice.

```java
// shared/error/DomainException.java
public abstract class DomainException extends RuntimeException {
    public abstract ErrorCode code();      // stable, documented, client-switchable
    public abstract HttpStatus status();
}

// One handler, RFC 7807 Problem Details, for the entire application
@RestControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(DomainException.class)
    ProblemDetail handle(DomainException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(ex.status());
        pd.setTitle(ex.code().title());
        pd.setDetail(ex.getMessage());
        pd.setProperty("code", ex.code().name());           // clients switch on this
        pd.setProperty("correlationId", MDC.get("correlationId"));
        return pd;
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail handleUnexpected(Exception ex) {
        log.error("Unhandled exception", ex);               // full detail to logs…
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.INTERNAL_SERVER_ERROR);
        pd.setDetail("An unexpected error occurred.");      // …nothing internal to the client
        pd.setProperty("correlationId", MDC.get("correlationId"));
        return pd;
    }
}
```

Rule ENG-3.2 — Never leak internals in an error response CRITICAL

Stack traces, SQL, class names and upstream provider messages never reach a client. The correlation ID is the bridge: the user reports it, and it locates the full detail in the logs. **Why:** error responses are the most reliable reconnaissance surface an attacker has, and the most common accidental disclosure path.

3.6 Validation — two distinct kinds

Kind	Where	What it checks
Input validation	Request DTO, Bean Validation annotations	Shape: required, length, format, range. Cheap, mechanical, rejects malformed requests with 400 before any logic runs.
Domain invariants	Inside the domain object, in its constructor or factory	Truths that must hold for the object to be valid at all: credits never negative, a session's end after its start, a booking's location matching its session's. Enforced at construction so an invalid instance cannot exist.

```java
// Domain invariant enforced at construction — an invalid PtPackage is unconstructible
public record PtPackage(PackageId id, UserId member, TrainerId trainer,
                        int creditsTotal, int creditsRemaining, LocalDate validUntil) {
    public PtPackage {
        if (creditsRemaining < 0)
            throw new IllegalArgumentException("credits cannot be negative");
        if (creditsRemaining > creditsTotal)
            throw new IllegalArgumentException("remaining cannot exceed total");
    }
    public PtPackage consumeOne() {
        if (creditsRemaining == 0) throw new NoCreditsRemainingException(id);
        return new PtPackage(id, member, trainer, creditsTotal, creditsRemaining - 1, validUntil);
    }
}
```

Note the database also carries `CHECK (credits_remaining >= 0)`. The redundancy is deliberate: the domain object gives a good error message, the constraint guarantees the rule even if a future code path forgets it.

3.7 Nullability, immutability and naming

Rule	Reason
`Optional` for return values, never for fields or parameters	Its purpose is expressing "may be absent" at a call site. As a field it adds indirection and breaks serialisation.
Records for DTOs, commands, events and value objects	Immutable by construction, concise, with correct `equals` for free. Use a class only when JPA requires one.
Typed identifiers, not raw `UUID`	`UserId` and `LocationId` are different types, so passing one where the other belongs will not compile. Given how many identifiers this domain juggles, this catches a real and otherwise silent class of bug.
No abbreviations in names	`membership`, not `mbrshp`. The cost is keystrokes; the benefit is every future reader, human or agent.
Boolean methods read as assertions	`isActive()`, `hasCreditsRemaining()`, `canBookAt(location)`.
Test names are sentences	`locationAdminCannotReadSiblingLocationMembers()`. Per `ENG-04`, the test list should read as a specification.

4. The Five Non-Negotiables

Each of these, if broken, produces a defect that compiles, passes ordinary tests, and shows no symptom until it matters. They are the reason this document exists.

> **NN-1 — Use `SET LOCAL`, never `SET`, for RLS context.**
> Connections are pooled. Plain `SET` persists on the connection after your request completes, so the next request — potentially serving a different gym — inherits your tenant context and reads their data. `SET LOCAL` is transaction-scoped and discarded on commit or rollback, which is the only reason pooling is safe here. Never write a bare `SET` against the application datasource.

> **NN-2 — The application never connects as the table owner.**
> A table owner bypasses RLS policies by default. If `lb_app` owns the tables, every policy is inert and the system appears to work perfectly while enforcing nothing. Two roles: `lb_migrator` owns the schema and runs Flyway; `lb_app` owns nothing and serves requests. Every table also carries `FORCE ROW LEVEL SECURITY`.

> **NN-3 — Sensitive media is never uploaded with Cloudinary's default type.**
> The default `upload` type yields a permanent, public, unrevocable URL. Applied to a receipt or an identity document that is a privacy breach which deleting the database row does not undo, because the URL may already be cached or shared. All uploads go through `MediaService`, which takes sensitivity as a required argument. There is no other upload path.

> **NN-4 — Store Cloudinary `public_id`, never a full URL.**
> URLs are constructed at read time. A stored URL embeds the provider in every row referencing an image, turning a provider change from a one-class edit into a data rewrite — and it bypasses the signing logic that `NN-3` depends on.

> **NN-5 — Every tenant-scoped service method authorises before it acts.**
> `auth.require(permission, locationId)` is the first statement. RLS is the second line of defence, not the first; relying on it alone means an authorisation bug degrades silently into "returns nothing" instead of failing loudly, and write paths are not protected by read policies at all.

4.1 The RLS context implementation

```
// shared/rls/RlsContextAspect.java — applied to every request-bound transaction
@Aspect @Component @RequiredArgsConstructor
public class RlsContextAspect {

    private final JdbcTemplate jdbc;
    private final AuthContextProvider authProvider;

    @Before("@annotation(org.springframework.transaction.annotation.Transactional)")
    public void applyContext() {
        AuthContext ctx = authProvider.current();

        // SET LOCAL — transaction-scoped. See NN-1. A bare SET here is a data breach.
        jdbc.update("SET LOCAL app.gym_id = ?",          ctx.gymId().value());
        jdbc.update("SET LOCAL app.location_ids = ?",    ctx.locationIdsCsv());
        jdbc.update("SET LOCAL app.gym_wide = ?",        ctx.isGymOwner());
        jdbc.update("SET LOCAL app.is_super_admin = ?", ctx.isSuperAdmin());
    }
}
```

> **Why missing context is safe.** The policies read `current_setting('app.gym_id', true)` — the `true` makes a missing setting return `NULL` rather than raising. A `NULL` comparison matches no rows, so a query that somehow escaped the aspect returns empty rather than unfiltered. That is `ENG-06` in concrete form: the failure mode is refusal, and it is immediately obvious in a test.

5. API Design Conventions

Concern	Convention
Base path	`/api/v1`. The version is in the path and increments only on a breaking change. Old app builds stay in circulation for weeks after a store release (`NFR-MNT-04`), so v1 must keep working long after v2 ships.
Resource naming	Plural nouns, kebab-case: `/check-ins`, `/pt-packages`. Never verbs — the method is the verb.
Location scoping	Location-scoped resources nest under their location: `/api/v1/locations/{locationId}/members`. This makes the scope explicit in the route and gives the service its authorisation argument directly, rather than inferring it.
State transitions	A sub-resource, not a verb endpoint: `POST /bookings/{id}/cancellation` rather than `/cancelBooking`.
Status codes	200 read, 201 create with `Location`, 204 delete, 400 malformed, 401 unauthenticated, 403 unauthorised, 404 absent *or invisible*, 409 state conflict, 422 valid but rejected by a business rule, 429 rate-limited.
404 versus 403	A resource outside the caller's location scope returns **404, not 403**. A 403 confirms the resource exists, which leaks the existence of another gym's data — the exact thing `NFR-SEC-01a` forbids.
Pagination	Cursor-based: `?cursor=&limit=`, returning `{ items, nextCursor }`. Offset pagination skips and duplicates rows as data shifts beneath it, which attendance and bookings do constantly.
Idempotency	`Idempotency-Key` header required on every `POST` that the mobile client may retry. The server stores it and returns the original result on replay.
Dates	ISO-8601 with offset, always UTC on the wire. Never a bare local date-time — a gym in a different timezone is a supported case.
Money	Integer minor units (paise) with an explicit currency. Never a floating-point type anywhere near money.
Naming in JSON	`camelCase`, matching TypeScript. Configured once on the serialiser, never per-field.

6. Database Standards

Rule	Detail
Naming	`snake_case`; plural tables (`check_ins`); singular columns; foreign keys `<entity>_id`; indexes `idx_<table>_<cols>`; constraints `ck_` / `uq_` / `fk_`.
Required columns	Every operational table: `id uuid` (UUIDv7), `gym_id uuid NOT NULL`, `location_id uuid NOT NULL`, `created_at`, `updated_at`, `created_by`. The scoping columns are not optional — they are what the RLS policies filter on.
Enums	Postgres enum types, not strings. Adding a value is a migration, which is correct: a new booking state should be a deliberate schema decision.
Soft delete	`deleted_at timestamptz` on entities `FR-ADM-03` covers. Never `DELETE` a member, payment or attendance row.
Migrations	Flyway, forward-only, one logical change each. `V<n>__<description>.sql`. A migration merged to main is immutable — corrections are new migrations.
Every new table	The same migration that creates a table must enable and force RLS and attach its policy. A table without a policy is a leak waiting to happen, so the two are never separated.
No business logic in the database	No triggers, no stored procedures — with one exception: constraints expressing invariants (`CHECK`, `UNIQUE`, foreign keys), which are declarations rather than logic and belong there.

```sql
-- The standard shape of a create-table migration. Both halves, always together.
CREATE TABLE check_ins (
    id              uuid        PRIMARY KEY,
    gym_id          uuid        NOT NULL REFERENCES gyms(id),
    location_id     uuid        NOT NULL REFERENCES locations(id),
    user_id         uuid        NOT NULL REFERENCES users(id),
    occurred_at     timestamptz NOT NULL,
    method          checkin_method  NOT NULL,
    purpose         checkin_purpose NOT NULL,
    idempotency_key text        NOT NULL,
    created_at      timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT uq_check_ins_idempotency UNIQUE (user_id, idempotency_key)
);

CREATE INDEX idx_check_ins_user_occurred  ON check_ins (user_id, occurred_at DESC);
CREATE INDEX idx_check_ins_gym_location   ON check_ins (gym_id, location_id);

-- Not optional, not a later migration, not a separate ticket:
ALTER TABLE check_ins ENABLE ROW LEVEL SECURITY;
ALTER TABLE check_ins FORCE  ROW LEVEL SECURITY;
CREATE POLICY p_check_ins_isolation ON check_ins USING (
    current_setting('app.is_super_admin', true)::boolean IS TRUE
    OR (gym_id = current_setting('app.gym_id', true)::uuid
        AND (current_setting('app.gym_wide', true)::boolean IS TRUE
             OR location_id = ANY(string_to_array(
                  current_setting('app.location_ids', true), ',')::uuid[])))
);
```

7. Security Coding Standards

Area	Standard
SQL	Parameterised queries or JPA only. String concatenation into SQL fails review unconditionally.
Passwords	Argon2id via Spring Security's encoder. Never a custom hash, never MD5 or SHA for credentials.
Tokens on device	`expo-secure-store` (Keychain / Keystore). Never `AsyncStorage`, which is unencrypted plaintext readable on a rooted device.
Secrets	Injected from Secrets Manager at runtime. Never in source, never in the image, never in `application.properties`. A committed secret is rotated, not deleted — git history is forever.
Rate limiting	Auth, OTP issuance and check-in are rate-limited per account and per device (`NFR-SEC-08`). OTP is the expensive one: each request costs real money.
QR payloads	Signed, time-limited, location-scoped. A photographed code must be useless later or elsewhere (`NFR-SEC-10`).
Dependencies	Dependabot plus OWASP dependency-check in CI, failing the build on high severity.
CORS	Explicit origin allowlist. Never `*` on an authenticated API.

> **Never logged, at any level, in any environment:** passwords, OTP codes, access or refresh tokens, full mobile numbers (last four digits only), health screening content, diet plan content, Cloudinary signed URLs, and API keys. Logs are shipped to third-party services, retained for months, and read by people who have no business seeing any of it. Log the *identifier*, never the *content*.

```
// WRONG — three separate disclosures
log.info("Login for {} with otp {}",
         user.phoneNumber(), otp);
log.debug("Health: {}", member.healthScreening());
```

```
// RIGHT — identifiers only
log.info("Login attempt userId={} phone=***{}",
         user.id(), user.phoneLast4());
log.debug("Health screening updated userId={}",
          member.id());
```

8. Mobile Standards — React Native and Expo

8.1 TypeScript

`strict: true`, plus `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes`. **`any` is banned** and fails lint; `unknown` with narrowing is the escape hatch. The generated API client is the single source of server types — never hand-write an interface that mirrors a server response, because the hand-written copy is what silently drifts.

8.2 State — three kinds, three tools

Kind	Tool	Examples and rule
Server state	TanStack Query	Anything the backend owns. Never copied into another store — a second copy is a second truth that will diverge.
Global client state	Zustand	Auth session, active role, theme. Deliberately tiny.
Local UI state	`useState`	Form fields, expanded rows, sheet visibility. Never promoted to global "just in case".

8.3 Every data screen handles four states

`UI-04` requires an explicit loading, empty, error and success state on every data-bearing view. Not a convention — a component contract, so it cannot be skipped by omission.

```
// features/attendance/useCheckInHistory.ts
export function useCheckInHistory(locationId: LocationId) {
  return useQuery({
    queryKey: ['check-ins', locationId],        // location in the key: never mix locations
    queryFn: () => api.checkIns.list({ locationId }),
    staleTime: 30_000,
  });
}

// The screen cannot forget a state — the component requires all four.
<QueryView
  query={useCheckInHistory(locationId)}
  loading={() => <CheckInSkeleton />}
  empty={() => <EmptyState title="No check-ins yet" />}
  error={(e, retry) => <ErrorState error={e} onRetry={retry} />}
  success={(data) => <CheckInList items={data.items} />}
/>
```

8.4 The offline outbox

Only check-in and workout logging are queued (ADR-06). The rule that makes this safe is worth restating: **both operations are non-contending** — a check-in is an append-only fact, a workout log has exactly one author — so no conflict resolution is needed. Anything that contends for a shared resource stays online.

```ts
// lib/sync/outbox.ts
export async function enqueue<T extends OutboxOperation>(op: T): Promise<void> {
  await db.insert(outbox).values({
    id: randomUUID(),
    idempotencyKey: randomUUID(),   // generated ONCE; reused on every retry
    type: op.type,
    payload: JSON.stringify(op.payload),
    occurredAt: new Date().toISOString(),   // original time preserved — FR-ATT-07
    attempts: 0,
  });
  void drain();                     // fire and forget; UI already confirmed
}

export async function drain(): Promise<void> {
  if (!(await isOnline())) return;

  for (const item of await pendingItems()) {
    try {
      // Same key every attempt — this is what makes retry safe (FR-ATT-05)
      await api.submit(item.type, JSON.parse(item.payload), {
        idempotencyKey: item.idempotencyKey,
      });
      await remove(item.id);
    } catch (e) {
      if (isPermanent(e)) { await markFailed(item.id, e); }   // 4xx: surface to user
      else { await backoff(item); break; }                    // 5xx/network: retry later
    }
  }
}
```

Rule ENG-8.1 — The idempotency key is generated once, at enqueue CRITICAL

Generating a fresh key per attempt defeats the entire mechanism: each retry looks like a new operation and creates a duplicate row. The key belongs to the *operation*, not to the *attempt*. This single line is what `FR-ATT-05` rests on under offline replay.

8.5 Media

All uploads go through `lib/media`, which requires a sensitivity argument (`NN-3`) and returns a `public_id` (`NN-4`). Display URLs are built with `f_auto,q_auto` and an explicit width, so a 12-megapixel exercise demonstration image is never downloaded whole onto a phone over mobile data. Media in scope is exercise and gym imagery, profile pictures, receipts and optional identity documents — **not** progress photographs, which are out of scope (§13).

9. Testing Strategy

9.1 What must be tested

`NFR-MNT-03` names five areas as minimum coverage. They are not suggestions — each corresponds to a failure that costs money or trust.

Area	Why it is mandatory	Type
Authorisation boundaries	Every role against every permission at an in-scope and an out-of-scope location. The matrix is large and entirely mechanical — exactly what tests are for.	Integration
Gym and location isolation	The evidence for `NFR-SEC-01a`. Requires real Postgres; RLS cannot be tested against an in-memory database.	Testcontainers
Billing arithmetic	Proration, GST, partial payments, dues, refunds. Money bugs are discovered by customers.	Unit
Credit decrementing	PT credits under concurrency, expiry, and cancellation-policy edges.	Integration
Booking concurrency	`NFR-PRF-05` in executable form: N threads, one remaining place, exactly one winner.	Integration

9.2 The isolation suite

```
@SpringBootTest
@Testcontainers
class LocationIsolationTest {

    @Container
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16");

    // Two gyms, two locations each — the fixture is the point. Isolation bugs
    // are invisible when only one tenant exists in the database.
    @BeforeEach void seed() { fixtures.twoGymsTwoLocationsEach(); }

    @Test
    void locationAdminCannotReadSiblingLocationMembers() {
        asLocationAdmin(GYM_A, LOCATION_A1);

        List<Member> visible = memberRepository.findAll();

        assertThat(visible).allMatch(m -> m.locationId().equals(LOCATION_A1));
        assertThat(visible).noneMatch(m -> m.locationId().equals(LOCATION_A2));
    }

    @Test
    void locationAdminCannotReadMemberFromAnotherGymEvenByDirectId() {
        asLocationAdmin(GYM_A, LOCATION_A1);
        // Direct lookup by a known id — the identifier-guessing path NFR-SEC-01a names
        assertThat(memberRepository.findById(KNOWN_MEMBER_IN_GYM_B)).isEmpty();
    }

    @Test
    void queryWithoutRlsContextReturnsNothing() {
        clearRlsContext();
        // Fails closed, per ENG-06. If this ever returns rows, RLS is misconfigured
        // and NN-2 is the first thing to check.
        assertThat(memberRepository.findAll()).isEmpty();
    }
}
```

> **This suite may never be disabled to make a build pass.** It is the only evidence that the platform's central security claim is true. If it fails, the correct response is to stop and fix the isolation defect — never to `@Disabled` it and open a ticket.

9.3 Concurrency test — the booking race

```
@Test
void concurrentBookingsNeverExceedCapacity() throws Exception {
    Session session = fixtures.sessionWithCapacity(1);          // one place left
    var latch = new CountDownLatch(1);

    List<Future<BookingResult>> results = new ArrayList<>();
    try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
        for (int i = 0; i < 20; i++) {
            UserId member = fixtures.member();
            results.add(pool.submit(() -> { latch.await();       // all start together
                return bookingService.book(session.id(), member); }));
        }
        latch.countDown();
    }

    assertThat(countOf(results, BOOKED)).isEqualTo(1);           // exactly one winner
    assertThat(countOf(results, WAITLISTED)).isEqualTo(19);      // no one lost
}
```

9.4 Test pyramid and conventions

Level	Share	Scope and speed
Unit	~60%	Domain objects and pure logic. No Spring context, no database. Milliseconds.
Integration	~30%	Service through to real Postgres via Testcontainers. Where authorisation, RLS and concurrency are proven.
Contract	~7%	Controller layer with MockMvc, asserting the OpenAPI contract and error shapes.
End-to-end	~3%	Detox on mobile, for the few flows whose breakage is catastrophic: sign-in, check-in, booking.

Coverage gate: 80% on service and domain packages; no gate on web or DTOs, where the number would measure nothing. Fixtures are builders with sensible defaults (`fixtures.member().at(LOCATION_A1).withActiveMembership()`) so a test states only what it actually depends on.

10. Automated Quality Gates

Tool	Enforces	Blocks
Spotless	Java formatting, import order, licence headers. Auto-applied; never discussed in review.	Yes
Checkstyle	Naming, method length, cyclomatic complexity, banned imports.	Yes
ArchUnit	Layering (§3.1): controllers may not reference repositories; services may not reference web types; repositories are package-private.	Yes
Spring Modulith	Module boundaries (§2.2): no access outside another module's `api/`.	Yes
Custom ArchUnit rules	The non-negotiables: no bare `SET`, no direct Cloudinary SDK use outside `MediaService`, no `@Transactional` on a controller, no entity returned from a controller.	Yes
JaCoCo	Coverage thresholds on service and domain packages.	Yes
OWASP check	Dependency vulnerabilities at high severity or above.	Yes
ESLint + Prettier	TypeScript rules, `no-explicit-any`, hooks rules, import order.	Yes
tsc --noEmit	Strict type checking including the generated client.	Yes

```
// backend/src/test/java/.../ArchitectureRulesTest.java — layering as executable rules
@AnalyzeClasses(packages = "com.limitbeyond", importOptions = DoNotIncludeTests.class)
class ArchitectureRulesTest {

    @ArchTest
    static final ArchRule controllersMustNotTouchRepositories =
        noClasses().that().resideInAPackage("..web..")
            .should().dependOnClassesThat().resideInAPackage("..persistence..");

    @ArchTest
    static final ArchRule servicesMustNotDependOnWebTypes =
        noClasses().that().resideInAPackage("..internal..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "jakarta.servlet..", "org.springframework.web..");

    @ArchTest  // NN-3 / NN-4: one upload path, no exceptions
    static final ArchRule onlyMediaServiceUsesCloudinary =
        noClasses().that().doNotHaveSimpleName("MediaService")
            .should().dependOnClassesThat().resideInAPackage("com.cloudinary..");

    @ArchTest  // NN-5: transactions (and therefore RLS context) start in services
    static final ArchRule noTransactionalControllers =
        noClasses().that().resideInAPackage("..web..")
            .should().beAnnotatedWith(Transactional.class);
}
```

11. Git Workflow and Code Review

Practice	Standard
Branching	Trunk-based. Short-lived branches off `main`, merged within a day or two. `main` is always deployable.
Branch names	`feat/`, `fix/`, `chore/`, `docs/` plus a short description.
Commits	Conventional Commits: `feat(attendance): add geofence validation`. The scope is the module. Enables generated changelogs and makes history greppable by area.
Pull requests	One logical change. A PR touching six modules is six PRs. Description states what and why; the diff already says how.
Requirement traceability	PRs implementing a requirement cite it (`Implements FR-ATT-02`). This is what makes the SRS auditable after the fact rather than aspirational.
Merging	Squash merge. One commit per logical change on `main`.
Self-review	Working solo, review your own PR the next morning before merging. The delay is what makes it a review rather than a re-read.

> **Reviewing as a solo developer.** Most review value here comes from the automated gates, which do not get tired or optimistic. The human pass should concentrate on what tools cannot judge: is the authorisation check correct and at the right scope, does this belong in this module, and will this read clearly in six months. Appendix A is ordered accordingly, with the security items first.

12. Observability Standards

Level	Use for
`ERROR`	Something failed that needs a human. Every occurrence should be actionable — if it is not, it is not an error.
`WARN`	Recovered degradation: an external service retried, a job re-queued, a rate limit hit.
`INFO`	Business events worth reconstructing later: check-in recorded, booking made, payment recorded, admin promoted.
`DEBUG`	Development only. Never enabled in production.

Every log line carries `correlationId`, `userId`, `gymId` and `locationId` from the MDC, populated by the same filter that builds the `AuthContext`. That combination is what satisfies `NFR-MNT-02`: diagnosing a failed check-in without reproducing it requires knowing who, which gym, which location, and which request — and all four are present on every line by construction rather than by remembering to include them.

```
// Structured, queryable, and free of content — see §7
log.info("Check-in recorded checkInId={} userId={} locationId={} method={} purpose={} deferred={}",
         checkIn.id(), checkIn.userId(), checkIn.locationId(),
         checkIn.method(), checkIn.purpose(), checkIn.isDeferred());
```

13. The Permission Matrix

The authoritative expression of who may do what. It resolves the Super Admin question directly: **the Super Admin has full operational authority across every gym, but no data access beyond what a Gym Owner holds within one.** Owning the platform confers reach, not depth.

● permitted · ◐ permitted within the account's own location scope · — denied · **Self** = on the account's own record only

Capability	Super Admin	Gym Owner	Location Admin	Trainer	Member
Verify / activate / suspend a gym	●	—	—	—	—
Record gym subscription payment	●	—	—	—	—
Cross-gym reporting	●	—	—	—	—
Manage gym profile, branding	●	●	—	—	—
Manage plan catalogue and pricing	●	●	—	—	—
Create / close locations	●	●	—	—	—
Appoint / demote Location Admins	●	●	◐	—	—
Verify trainers and members	●	●	◐	—	—
Manage members; assign trainers	●	●	◐	—	—
Record member payments; issue receipts	●	●	◐	—	—
Build timetables; manage sessions	●	●	◐	—	—
Mark attendance for others	●	●	◐	◐	—
Author workout and diet plans	●	●	◐	◐	—
Conduct and log PT sessions	●	●	◐	◐	—
Issue coaching feedback	●	●	◐	◐	—
View health screening data	◐	◐	◐	◐	Self
View body measurements, weight	◐	◐	◐	◐	Self
Own personal fitness profile	●	●	●	●	●

> **The shaded row, explained. Health screening** is visible to admins because it is operationally necessary — `FR-MEM-03` collects injuries and conditions precisely so that whoever programmes a member's training does not hurt them. Access is scoped to the member's own location and is audited. Measurements and weight follow the same rule for the same reason: a trainer cannot programme progressively without them.

> **Progress photographs are out of scope.** They were the platform's most sensitive data class, they had no operational purpose, and they were the sole cause of a contradiction between `NFR-PRV-02` and `NFR-PRV-03`. Removing them resolves that contradiction outright rather than arbitrating it, and it takes with it the App Store review risk (`ARK-05`), the bulk of the DPDP exposure, and the need for any break-glass access path. Progress tracking keeps weight, body measurements, body fat, personal records, volume trends and streaks — everything that actually drives the analytics in `M13`.
>
> Sensitive media handling does not disappear, only shrinks: receipts and optional identity documents (`FR-MEM-07`) still require authenticated Cloudinary delivery. `NN-3` and `NN-4` stand unchanged, applied to a smaller and far less alarming set. Should photographs be wanted later, they return as a new requirement with this matrix as the starting point — not as a quiet re-enablement.

Appendix A — Code Review Checklist

Ordered by consequence. If time is short, the first block is the block that matters.

SECURITY AND CORRECTNESS — NEVER SKIPPED

- ☐ Does every tenant-scoped service method call `auth.require(...)` with the *correct location*, as its first statement? (`NN-5`)
- ☐ Any bare `SET` against the app datasource? Must be `SET LOCAL`. (`NN-1`)
- ☐ New table: does the same migration enable *and force* RLS and attach a policy? (`NN-2`)
- ☐ Media upload: sensitivity argument passed, and is it right? Storing `public_id`, not a URL? (`NN-3`, `NN-4`)
- ☐ Retryable `POST`: is the idempotency key generated once per operation, not per attempt? (`ENG-8.1`)
- ☐ Contended row (capacity, credits): is it locked, and is there a concurrency test?
- ☐ Any PII, token, OTP or health content in a log line or error response? (§7)
- ☐ Out-of-scope resource: returns 404, not 403? (§5)
- ☐ External service call inside a transaction? (§3.4)

DESIGN

- ☐ Does this belong in this module, or is it here because it was convenient?
- ☐ Layering respected — no repository in a controller, no web type in a service?
- ☐ Domain invariants in the domain object rather than scattered through the service?
- ☐ Reusing the existing error model, pagination shape and job queue rather than a new variant? (`ENG-05`)

TESTS AND CLARITY

- ☐ Do the tests cover the failure cases, not only the happy path?
- ☐ Do test names read as specifications? (`ENG-04`)
- ☐ Would this be clear to someone reading it in six months with no context?
- ☐ Requirement cited in the PR description where one applies?

Appendix B — Definition of Done

A change is done when all of the following are true. Not most.

1. Implements the requirement it cites, including its edge cases.
2. All automated gates pass — no exclusions, no `@Disabled`, no suppressed warnings without a comment saying why.
3. Tests exist for the failure paths, not just the success path.
4. Where it touches tenant data, an authorisation test exists for both an in-scope and an out-of-scope caller.
5. Where it adds a table, the isolation suite covers it.
6. Logs carry the standard fields and leak no content.
7. The API change is reflected in OpenAPI and the client is regenerated.

8. Any architectural decision taken along the way is recorded as an ADR.
9. Any SRS contradiction discovered is recorded in Appendix D rather than resolved silently in code.

Appendix C — `CLAUDE.md` Agent Ruleset

This file lives at the repository root. AI coding assistants read it automatically and follow it as project instructions. It deliberately front-loads the non-negotiables, because those are the rules an agent would otherwise get wrong *plausibly* — producing code that looks idiomatic, compiles, passes a happy-path test, and quietly breaks a security boundary.

```
# LIMITBEYOND — Project Instructions

Java 21 + Spring Boot (modular monolith) · PostgreSQL 16 with RLS ·
React Native + Expo · Cloudinary · Neon (dev, ap-southeast-1)

Specs: docs/LIMITBEYOND_SRS_v1.1.pdf · docs/LIMITBEYOND_ARCH_v1.0.pdf ·
       docs/LIMITBEYOND_STANDARDS_v1.0.pdf

## NEVER (these fail silently — no exception, no test failure, no symptom)

1. NEVER use bare `SET` for RLS context. Always `SET LOCAL`, inside a transaction.
   Plain SET persists on the pooled connection and leaks tenant context to the
   next request. This is a data breach, not a style issue.
2. NEVER connect the application as the table owner. `lb_app` owns nothing;
   `lb_migrator` owns the schema. Owners bypass RLS unless FORCE is set.
3. NEVER upload media with Cloudinary's default `upload` type when it may be
   sensitive. Receipts and ID documents use `authenticated`. All uploads go
   through MediaService, which requires a sensitivity argument.
4. NEVER store a full Cloudinary URL. Store `public_id`; build URLs at read time.
5. NEVER write a tenant-scoped service method without `auth.require(permission,
   locationId)` as its first statement.
6. NEVER log passwords, OTPs, tokens, full phone numbers, health screening or
   diet content. Log identifiers, never content.
7. NEVER return 403 for a resource outside the caller's scope — return 404.
   A 403 confirms existence and leaks another gym's data.
8. NEVER call an external service (Twilio, Cloudinary, Expo) inside a
   transaction. Enqueue a job instead.
9. NEVER generate a fresh idempotency key per retry. One key per operation.
10. NEVER use floating point for money. Integer minor units (paise) + currency.

## ALWAYS

- Create tables WITH their RLS policy in the same migration. Enable AND force.
- Include `gym_id` and `location_id` on every operational table.
- Put `@Transactional` on service methods only — never controllers, never repos.
- Enqueue jobs inside the same transaction as the change that triggers them.
- Lock contended rows (session capacity, PT credits) with `lockForUpdate`.
- Enforce domain invariants in the domain object's constructor.
- Give every data screen loading / empty / error / success states.
- Name tests as sentences: `locationAdminCannotReadSiblingLocationMembers`.

## Module structure

Ten modules: platform, identity, org, people, billing, attendance,
scheduling, training, engagement, insights (+ shared).

Only `api/` is public. `web/`, `domain/`, `internal/`, `persistence/` are
package-private. Cross-module access goes through the other module's facade
interface or a published event. Modulith verification fails the build otherwise.

`training` serves EVERY role, not just members — that is FR-PFP-01. No method
in it takes a role parameter or branches on one. A Gym Owner logging their own
sets uses the identical code path a member does.

## Permissions

Five roles: SUPER_ADMIN, GYM_OWNER, LOCATION_ADMIN, TRAINER, MEMBER.
Authority is a set of grants, each scoped to a gym and named locations.
Effective authority = union of grants EVALUATED PER LOCATION — never the
maximum applied everywhere.

Super Admin has full operational reach across all gyms but NO data access
beyond a Gym Owner's. Owning the platform confers reach, not depth.

Progress PHOTOGRAPHS are out of scope — do not build upload, storage or
display for them. Progress tracking is weight, measurements, body fat,
```

```
personal records, volume trends and streaks only.

## When unsure

Check the specs before inventing. If a requirement seems contradictory, say so
and ask — do not resolve it silently in code. If a rule above blocks something
that seems necessary, that is the signal to ask, not to work around it.
```

Appendix D — SRS Amendments Required

Contradictions found while writing this guide. Per your instruction the SRS stays at v1.1 for now; these accumulate here for the v1.2 pass, joined by the items already listed in LB-ARCH-1.0 Appendix B.

Requirement	Problem	Proposed resolution
FR-PRG-03	**Withdrawn.** Progress photographs are removed from scope. The requirement, its member-controlled trainer visibility setting, and all upload and display behaviour go with it.	Mark withdrawn in v1.2, in the same style as `FR-BIL-15`. Record the reason: no operational purpose, disproportionate privacy exposure, and the sole cause of the `NFR-PRV` contradiction below.
NFR-PRV-02 **NFR-PRV-03**	**Contradiction, now resolved by removal.** PRV-02 granted the tenant Admin access to health data, measurements *and* photographs; PRV-03 as revised in v1.1 stated that no administrative role had routine access to either. With photographs gone, the conflict reduces to a simple, answerable question about health data.	Amend both to a single consistent rule: health screening and measurements are visible to admins and the assigned trainer within the member's location scope, audited, because programming training safely requires them. Delete every reference to photographs.
NFR-PRV-03	Describes Super Admin access as available with "explicit, audited justification" — a break-glass mechanism that is now not being built.	Revise: Super Admin authority is capped at the Gym Owner data tier, with no elevated path. Simpler to build, simpler to state to a customer, and strictly safer.
FR-PRG-06 **M13**	Reference the photograph visibility setting and photograph comparison (`FR-PRG-07`).	Remove the photographic elements. Progress tracking retains weight, measurements, body fat, personal records, volume trends, streaks and goals — the full analytics substance of `M13` is unaffected.
ARK-05 (LB-ARCH)	App Store rejection risk was driven largely by health-adjacent imagery.	Downgrade materially. Health screening and location permission remain, but the highest-scrutiny element is gone.
FR-EXR-05	Carried over from LB-ARCH-1.0 Appendix B: requires porting the predecessor's exercise data, which ADR-11 replaced with a clean-start authored catalogue.	Revise to specify a freshly authored catalogue against the richer model.

Requirement	Problem	Proposed resolution
OPN-04	Still open, now build-blocking (`AOP-02`). RLS context assumes one active `gym_id` per request; a person belonging to two gyms needs a gym-switching mechanism in the token.	Decide before the `identity` module is implemented.

> **Keeping this document honest.** A standards document decays the moment it describes something the code no longer does. Two habits prevent it: when a rule is found to be wrong, change it here in the same PR that changes the code — never leave the document describing the old way. And when a rule turns out to be unenforceable by tooling, ask whether it should exist at all (`ENG-02`). A short document that is entirely true beats a thorough one that is half-obeyed.

— END OF DOCUMENT —